# Data Structures - Simple C++ Member Functions

[illegible]: [illegible] vector[illegible] [illegible] member functions [illegible].
Each type has 10 functions. Each function has 3 test cases.

# 1) ArrayList (Unsorted) - 10 Functions (Simple Member Functions)

### *1. insertEnd*

```
class ArrayList {
public:
    int a[100], n;
    ArrayList() { n = 0; }

    void insertEnd(int x) {
        if (n < 100) {
            a[n] = x;
            n++;
        }
    }
};
```

| Case | Input | Expected |
|---|---|---|
| 1 | n=0, insertEnd(5) | [5] |
| 2 | n=99, insertEnd(7) | item added at last |
| 3 | n=100, insertEnd(9) | no change |

### *2. insertAt*

```
void insertAt(int idx, int x) {
    if (idx < 0 || idx > n || n == 100) return;
    for (int i = n; i > idx; i--) a[i] = a[i - 1];
    a[idx] = x;
    n++;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,3,4], idx=1, x=2 | [1,2,3,4] |
| 2 | idx=0, x=9 | insert in start |
| 3 | idx>n | no change |

### *3. deleteAt*

```
void deleteAt(int idx) {
    if (idx < 0 || idx >= n) return;
    for (int i = idx; i < n - 1; i++) a[i] = a[i + 1];
    n--;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,3], idx=1 | [1,3] |
| 2 | idx=0 | remove first |
| 3 | idx invalid | no change |

### *4. findIndex*

```
int findIndex(int x) {
    for (int i = 0; i < n; i++) {
        if (a[i] == x) return i;
    }
    return -1;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[4,7,2], x=7 | 1 |
| 2 | x=9 | -1 |

| | | |
|---|---|---|
| 3 | empty list | -1 |

### 5. countValue

```
int countValue(int x) {
    int c = 0;
    for (int i = 0; i < n; i++) if (a[i] == x) c++;
    return c;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,1,1], x=1 | 3 |
| 2 | x not found | 0 |
| 3 | empty | 0 |

### 6. reverseList

```
void reverseList() {
    int l = 0, r = n - 1;
    while (l < r) {
        int t = a[l];
        a[l] = a[r];
        a[r] = t;
        l++; r--;
    }
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,3,4] | [4,3,2,1] |
| 2 | a=[5] | [5] |
| 3 | empty | empty |

### 7. maxValue

```
int maxValue() {
    if (n == 0) return -1;
    int mx = a[0];
    for (int i = 1; i < n; i++) if (a[i] > mx) mx = a[i];
    return mx;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[3,9,2] | 9 |
| 2 | a=[-1,-7,-3] | -1 |
| 3 | empty | -1 |

### 8. sumAll

```
int sumAll() {
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i];
    return s;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,3] | 6 |
| 2 | a=[5] | 5 |
| 3 | empty | 0 |

### 9. moveZerosToEnd

```
void moveZerosToEnd() {
    int b[100], k = 0;
```

```
    for (int i = 0; i < n; i++) if (a[i] != 0) b[k++] = a[i];
    while (k < n) b[k++] = 0;
    for (int i = 0; i < n; i++) a[i] = b[i];
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,0,2,0,3] | [1,2,3,0,0] |
| 2 | a=[0,0] | [0,0] |
| 3 | a=[4,5] | [4,5] |

## *10. removeDuplicatesSimple*

```
void removeDuplicatesSimple() {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; ) {
            if (a[j] == a[i]) {
                for (int k = j; k < n - 1; k++) a[k] = a[k + 1];
                n--;
            } else j++;
        }
    }
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,1,3,2] | [1,2,3] |
| 2 | a=[7,7,7] | [7] |
| 3 | a=[1,2,3] | same |

# 2) ArrayList (Sorted) - 10 Functions (Simple Member Functions)

### *1. binarySearch*

```
int binarySearch(int x) {
    int l = 0, r = n - 1;
    while (l <= r) {
        int m = (l + r) / 2;
        if (a[m] == x) return m;
        if (a[m] < x) l = m + 1;
        else r = m - 1;
    }
    return -1;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,3,5,7], x=5 | 2 |
| 2 | x=2 | -1 |
| 3 | empty | -1 |

### *2. insertSorted*

```
void insertSorted(int x) {
    if (n == 100) return;
    int i = n - 1;
    while (i >= 0 && a[i] > x) {
        a[i + 1] = a[i];
        i--;
    }
    a[i + 1] = x;
    n++;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,3,5], x=4 | [1,3,4,5] |
| 2 | x=0 | insert first |
| 3 | x=9 | insert last |

### *3. deleteValue*

```
void deleteValue(int x) {
    int idx = binarySearch(x);
    if (idx == -1) return;
    for (int i = idx; i < n - 1; i++) a[i] = a[i + 1];
    n--;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,3], x=2 | [1,3] |
| 2 | x not found | same |
| 3 | single element match | empty |

### *4. firstIndexOf*

```
int firstIndexOf(int x) {
    for (int i = 0; i < n; i++)
        if (a[i] == x) return i;
    return -1;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,2,2], x=2 | 1 |

| | | |
|---|---|---|
| 2 | x=9 | -1 |
| 3 | empty | -1 |

### 5. lastIndexOf

```
int lastIndexOf(int x) {
    for (int i = n - 1; i >= 0; i--)
        if (a[i] == x) return i;
    return -1;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,2,2], x=2 | 3 |
| 2 | x=9 | -1 |
| 3 | empty | -1 |

### 6. countInRange

```
int countInRange(int L, int R) {
    int c = 0;
    for (int i = 0; i < n; i++)
        if (a[i] >= L && a[i] <= R) c++;
    return c;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,3,4,5], [2,4] | 3 |
| 2 | [6,9] | 0 |
| 3 | empty | 0 |

### 7. mergeTwoSorted

```
void mergeTwoSorted(int b[], int m, int out[], int &k) {
    int i = 0, j = 0; k = 0;
    while (i < n && j < m) {
        if (a[i] <= b[j]) out[k++] = a[i++];
        else out[k++] = b[j++];
    }
    while (i < n) out[k++] = a[i++];
    while (j < m) out[k++] = b[j++];
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,3,5], b=[2,4,6] | out=[1,2,3,4,5,6] |
| 2 | a empty | out=b |
| 3 | b empty | out=a |

### 8. removeAllValue

```
void removeAllValue(int x) {
    int w = 0;
    for (int i = 0; i < n; i++) if (a[i] != x) a[w++] = a[i];
    n = w;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,2,3], x=2 | [1,3] |
| 2 | x not found | same |
| 3 | all equal x | empty |

### 9. isSorted

```
bool isSorted() {
    for (int i = 1; i < n; i++)
        if (a[i] < a[i - 1]) return false;
    return true;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,2,3] | true |
| 2 | a=[1,3,2] | false |
| 3 | empty | true |

## *10. closestValue*

```
int closestValue(int x) {
    if (n == 0) return -1;
    int best = a[0];
    for (int i = 1; i < n; i++) {
        int d1 = a[i] - x; if (d1 < 0) d1 = -d1;
        int d2 = best - x; if (d2 < 0) d2 = -d2;
        if (d1 < d2) best = a[i];
    }
    return best;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | a=[1,4,6,8], x=5 | 4 or 6 |
| 2 | x=0 | 1 |
| 3 | empty | -1 |

# 3) LinkedList (Singly) - 10 Functions (Simple Member Functions)

### *1. pushFront*

```
class Node {
public:
    int data;
    Node* next;
    Node(int v) { data = v; next = NULL; }
};

class LinkedList {
public:
    Node* head;
    LinkedList() { head = NULL; }

    void pushFront(int x) {
        Node* p = new Node(x);
        p->next = head;
        head = p;
    }
};
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty, pushFront(5) | 5 |
| 2 | 1->2, pushFront(9) | 9->1->2 |
| 3 | pushFront twice | new nodes at front |

### *2. pushBack*

```
void pushBack(int x) {
    Node* p = new Node(x);
    if (head == NULL) { head = p; return; }
    Node* cur = head;
    while (cur->next != NULL) cur = cur->next;
    cur->next = p;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty + 4 | 4 |
| 2 | 1->2 + 3 | 1->2->3 |
| 3 | single + 9 | append end |

### *3. deleteFront*

```
void deleteFront() {
    if (head == NULL) return;
    Node* t = head;
    head = head->next;
    delete t;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->2->3 | 2->3 |
| 2 | single node | empty |
| 3 | empty | empty |

### *4. deleteByValue*

```
void deleteByValue(int x) {
    if (head == NULL) return;
    if (head->data == x) { deleteFront(); return; }
    Node* cur = head;
```

```
    while (cur->next != NULL && cur->next->data != x) cur = cur->next;
    if (cur->next != NULL) {
        Node* t = cur->next;
        cur->next = t->next;
        delete t;
    }
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->2->3, x=2 | 1->3 |
| 2 | x not found | same |
| 3 | delete head value | head removed |

## 5. search

```
bool search(int x) {
    Node* cur = head;
    while (cur != NULL) {
        if (cur->data == x) return true;
        cur = cur->next;
    }
    return false;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->4->6, x=4 | true |
| 2 | x=9 | false |
| 3 | empty | false |

## 6. length

```
int length() {
    int c = 0;
    Node* cur = head;
    while (cur != NULL) { c++; cur = cur->next; }
    return c;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->2->3 | 3 |
| 2 | single | 1 |
| 3 | empty | 0 |

## 7. getMiddle

```
int getMiddle() {
    if (head == NULL) return -1;
    Node *slow = head, *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow->data;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->2->3->4->5 | 3 |
| 2 | 1->2->3->4 | 3 |
| 3 | empty | -1 |

## 8. reverseList

```
void reverseList() {
    Node *prev = NULL, *cur = head;
    while (cur != NULL) {
        Node* nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    head = prev;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->2->3 | 3->2->1 |
| 2 | single | same |
| 3 | empty | empty |

## 9. countValue

```
int countValue(int x) {
    int c = 0;
    Node* cur = head;
    while (cur != NULL) {
        if (cur->data == x) c++;
        cur = cur->next;
    }
    return c;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->2->1->1, x=1 | 3 |
| 2 | x missing | 0 |
| 3 | empty | 0 |

## 10. removeDuplicatesSorted

```
void removeDuplicatesSorted() {
    Node* cur = head;
    while (cur != NULL && cur->next != NULL) {
        if (cur->data == cur->next->data) {
            Node* t = cur->next;
            cur->next = t->next;
            delete t;
        } else cur = cur->next;
    }
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | 1->1->2->3->3 | 1->2->3 |
| 2 | 1->1->1 | 1 |
| 3 | 1->2->3 | same |

# 4) Stack (Array Implementation) - 10 Functions

### *1. push*

```
class StackArr {
public:
    int a[100], top;
    StackArr() { top = -1; }

    bool push(int x) {
        if (top == 99) return false;
        top++; a[top] = x;
        return true;
    }
};
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty push 5 | top=0, a[0]=5 |
| 2 | full stack | false |
| 3 | push many | LIFO order |

### *2. pop*

```
bool pop() {
    if (top == -1) return false;
    top--;
    return true;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | stack [1,2,3] | remove 3 |
| 2 | single element | becomes empty |
| 3 | empty | false |

### *3. peek*

```
int peek() {
    if (top == -1) return -1;
    return a[top];
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | stack [4,9] | 9 |
| 2 | single [7] | 7 |
| 3 | empty | -1 |

### *4. isEmpty*

```
bool isEmpty() {
    return top == -1;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty | true |
| 2 | after push | false |
| 3 | after pop all | true |

### *5. size*

```
int size() {
    return top + 1;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty | 0 |
| 2 | [1,2,3] | 3 |
| 3 | after one pop | decrease by 1 |

### 6. clear

```
void clear() {
    top = -1;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | stack with items | becomes empty |
| 2 | already empty | stays empty |
| 3 | size after clear | 0 |

### 7. reverseString

```
void reverseString(char s[]) {
    StackArr st;
    int i = 0;
    while (s[i] != '\0') { st.push((int)s[i]); i++; }
    i = 0;
    while (!st.isEmpty()) { s[i++] = (char)st.peek(); st.pop(); }
    s[i] = '\0';
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | "abcd" | "dcba" |
| 2 | "a" | "a" |
| 3 | "" | "" |

### 8. isBalancedSimple

```
bool isBalancedSimple(char s[]) {
    StackArr st;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == '(') st.push('(');
        else if (s[i] == ')') {
            if (st.isEmpty()) return false;
            st.pop();
        }
    }
    return st.isEmpty();
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | "(())" | true |
| 2 | "(()" | false |
| 3 | ")(" | false |

### 9. toBinary

```
void toBinary(int x, char out[]) {
    StackArr st;
    if (x == 0) { out[0] = '0'; out[1] = '\0'; return; }
    while (x > 0) { st.push((x % 2) + '0'); x /= 2; }
    int i = 0;
    while (!st.isEmpty()) { out[i++] = (char)st.peek(); st.pop(); }
    out[i] = '\0';
}
```

| Case | Input | Expected |
|---|---|---|

| | | |
|---|---|---|
| 1 | x=10 | "1010" |
| 2 | x=0 | "0" |
| 3 | x=1 | "1" |

## *10. insertBottomRec*

```
void insertBottomRec(int x) {
    if (isEmpty()) { push(x); return; }
    int t = peek(); pop();
    insertBottomRec(x);
    push(t);
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | stack [1,2,3], x=9 | [9,1,2,3] |
| 2 | empty, x=5 | [5] |
| 3 | single [7], x=2 | [2,7] |

# 5) Queue (Array Implementation) - 10 Functions

### *1. enqueue*

```
class QueueArr {
public:
    int a[100], front, rear, cnt;
    QueueArr() { front = 0; rear = -1; cnt = 0; }

    bool enqueue(int x) {
        if (cnt == 100) return false;
        rear = (rear + 1) % 100;
        a[rear] = x;
        cnt++;
        return true;
    }
};
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty enqueue 5 | front=0 rear=0 |
| 2 | fill queue | cnt=100 |
| 3 | enqueue when full | false |

### *2. dequeue*

```
bool dequeue() {
    if (cnt == 0) return false;
    front = (front + 1) % 100;
    cnt--;
    return true;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | queue [1,2,3] | remove 1 |
| 2 | single element | becomes empty |
| 3 | empty | false |

### *3. getFront*

```
int getFront() {
    if (cnt == 0) return -1;
    return a[front];
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | queue [7,8] | 7 |
| 2 | single [5] | 5 |
| 3 | empty | -1 |

### *4. isEmpty*

```
bool isEmpty() {
    return cnt == 0;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty | true |
| 2 | after enqueue | false |
| 3 | after dequeue all | true |

### *5. size*

```
int size() {
    return cnt;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | empty | 0 |
| 2 | [1,2,3] | 3 |
| 3 | after one dequeue | 2 |

### 6. reverseQueueSimple

```
void reverseQueueSimple() {
    int b[100], k = 0;
    while (!isEmpty()) { b[k++] = getFront(); dequeue(); }
    for (int i = k - 1; i >= 0; i--) enqueue(b[i]);
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | q=[1,2,3,4] | [4,3,2,1] |
| 2 | single | same |
| 3 | empty | empty |

### 7. rotateLeft

```
void rotateLeft(int k) {
    if (cnt == 0) return;
    k = k % cnt;
    while (k--) {
        int x = getFront();
        dequeue();
        enqueue(x);
    }
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | q=[1,2,3,4], k=1 | [2,3,4,1] |
| 2 | k=4 | same |
| 3 | single, k=10 | same |

### 8. clear

```
void clear() {
    front = 0;
    rear = -1;
    cnt = 0;
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | queue with items | empty |
| 2 | already empty | empty |
| 3 | size after clear | 0 |

### 9. countValue

```
int countValue(int x) {
    int c = 0;
    for (int i = 0; i < cnt; i++) {
        int idx = (front + i) % 100;
        if (a[idx] == x) c++;
    }
    return c;
}
```

| Case | Input | Expected |
|---|---|---|

| | | |
|---|---|---|
| 1 | q=[1,2,1,1], x=1 | 3 |
| 2 | x missing | 0 |
| 3 | empty | 0 |

## *10. firstNegativeWindow3*

```
void firstNegativeWindow3(int in[], int n, int out[]) {
    for (int i = 0; i <= n - 3; i++) {
        out[i] = 0;
        for (int j = i; j < i + 3; j++) {
            if (in[j] < 0) { out[i] = in[j]; break; }
        }
    }
}
```

| Case | Input | Expected |
|---|---|---|
| 1 | in=[12,-1,-7,8,-15], k=3 | [-1,-1,-7] |
| 2 | in=[1,2,3,4] | [0,0] |
| 3 | in=[-5,-2,-3] | [-5] |